Revisão P1
Construção de Compiladores
Prof. Bráulio de Mello

Erickson G. Müller

23 de Abril de 2026

Conteúdos

- Construção de Analisadores Léxicos
- Reconhecimento Sintático por APND
- APND, Analisador Sintático Preditivo
- Analisadores Left-Right
  - Construção de tabela de parsing SLR
  - Algoritmo de mapeamento do parser LR para análise sintática.

1 Construção de Analisador Léxico

A saída do Analisador Léxico é a entrada do Analisador Sintático.
Processo:

1. Definir tokens
2. Construir o AFND
3. Determinar o AF
4. Acrescentar estado de erro para tratar tokens inválidos
5. Construir o analisador (algoritmo)
6. Tratamento de erro
   (a) Rótulo não reconhecido
   (b) Descrição do erro
   (c) Adicionar na Tabela de Símbolos
7. Reconhecimento

1 estado reconhece 1 token
1 Token pode ser reconhecido por mais de 1 estado

1.1 Tabela de Símbolos

A tabela de símbolos segue a estrutura:
Fita: [sequência de estados]+$
L1: rótulo da cadeia
[Mensagem de erro]

2 Reconhecedores Sintáticos

- GLC
- Autômatos de PIlha
- APND
  - K - conjunto de estados
  - $\sum$ - alfabeto de entrada

- Υ - alfabeto da pilha
- δ - função de transição (*atua sobre a fita*)
- i - estado inicial ($i \in K$)
- I - símbolo inicial da pilha ($I \in \Upsilon$)
- F - conjunto de estados finais ($F \subseteq K$)

Se $(p, \alpha) \in \delta(q, a, Z)$ então $[q, ax, zy] \vdash^* [p, x, \alpha y]$.
* $\vdash$ = *transição/fecho transitivo*
* α = *símbolo empilhamento*
* a = *início da fita*
* Z = *topo da pilha*

2.1 Aceitação por estado final

$L_{ef}(A) = \{x \in \sum^* | [i, x, I] \vdash^* [f, \varepsilon, y]$ e $f \in F$ e $y \in \Upsilon^*$ qualquer$\}$

2.2 Aceitação por pilha vazia

$L_{pv}(A) = \{x \in \sum^* | [i, x, I] \vdash^* [f, \varepsilon, \varepsilon]$ e $f \in K$ qualquer$\}$

2.3 Exemplo

$K = \{0, 1, 2\}$
$i = 0$
$\sum = \{a, b\}$
$I = X$
$\Upsilon = \{X, A\}$
$F = \{2\}$
Funções de Transição

$$1 : \delta(0, a, X) = \{(0, AX)\}$$
$$2 : \delta(0, a, A) = \{(0, AA)\}$$
$$3 : \delta(0, b, A) = \{(1, \varepsilon)\}$$
$$4 : \delta(1, b, A) = \{(1, \varepsilon)\}$$
$$5 : \delta(1, \varepsilon, X) = \{(2, X)\}$$

Fita Resultada do analisador léxico:
Fita: $[0, aaabbb, X] \vdash$

* *Lembrar que estado inicial é 0 e X está no topo da pilha.*

$1^{\underline{a}}$ transição: $[0, aaabbb, X] \vdash [0, aabbb, AX]$

$2^{\underline{a}}$ transição: $[0, aaabbb, X] \vdash [0, aabbb, AX] \vdash [0, abbb, AAX]$

$3^{\underline{a}}$ transição: $[0, aaabbb, X] \vdash [0, aabbb, AX] \vdash [0, abbb, AAX] \vdash [0, bbb, AAAX]$

$4^{\underline{a}}$ transição: $[0, aaabbb, X] \vdash [0, aabbb, AX] \vdash [0, abbb, AAX] \vdash [0, bbb, AAAX] \vdash [1, bb, AAX]$

$5^{\underline{a}}$ transição: $[0, aaabbb, X] \vdash [0, aabbb, AX] \vdash [0, abbb, AAX] \vdash [0, bbb, AAAX] \vdash [1, bb, AAX] \vdash [1, b, AX]$

$6^{\underline{a}}$ transição: $[0, aaabbb, X] \vdash [0, aabbb, AX] \vdash [0, abbb, AAX] \vdash [0, bbb, AAAX] \vdash [1, bb, AAX] \vdash [1, b, AX] \vdash [1, \varepsilon, X]$

$7^{\underline{a}}$ transição: $[0, aaabbb, X] \vdash [0, aabbb, AX] \vdash [0, abbb, AAX] \vdash [0, bbb, AAAX] \vdash [1, bb, AAX] \vdash [1, b, AX] \vdash [1, \varepsilon, X] \vdash [2, \varepsilon, X]$

Assim, $[2, \varepsilon, X] \in F$

2.4 Exemplo 2

$K = \{0, 1\}$
$i = \{0\}$
$\sum = \{a, b\}$
$I = \{X\}$
$\Upsilon = \{X, A\}$
$F = \emptyset$

Funções de Transição

$$1 : \delta(0, a, X) = \{(0, A)\}$$

$$2 : \delta(0, a, A) = \{(0, AA)\}$$

$$3 : \delta(0, b, a) = \{(1, \varepsilon)\}$$

$$4 : \delta(1, b, A) = \{(1, \varepsilon)\}$$

Fita: $[0, aabbb, X] \vdash [0, abbb, A] \vdash [0, bbb, AA] \vdash [1, bb, A] \vdash [1, b, \varepsilon]$ **não ocorreu reconhecimento no último estado**

2.5 Exemplo Gramática de Operadores

$E ::= E + T|T$
$T ::= T * F|F$
$F ::= (E)|a$

Reconhecimento por pilha vazia

Fita:$[0, a * (a + a), E] \vdash$

Criação de regras:

$$1 : \delta(0, \varepsilon, E) = \{(0, T)\}$$

Comentário: o autômato de pilha é não determinístico pois não podemos escolher qual produção da regra será usado na transição para reduzir e acrescentar no topo da pilha. A regra 1 reduz uma produção de E e empilha uma produção de T

Fita:$[0, a * (a + a), E] \vdash [0, a * (0 + a), T] \vdash$

Criação de nova regra:

$$1 : \delta(0, \varepsilon, E) = \{(0, T)\}$$

$$2 : \delta(0, \varepsilon, T) = \{(0, T * F), (0, F)\}$$

Fita:$[0, a * (a + a), E] \vdash [0, a * (0 + a), T] \vdash [0, a * (a + a), T * F] \vdash [0, a * (a + a), F * F] \vdash$

Criação de nova regra: Criação de nova regra:

$$1 : \delta(0, \varepsilon, E) = \{(0, T)\}$$

$$2 : \delta(0, \varepsilon, T) = \{(0, T * F), (0, F)\}$$

$$3 : \delta(0, \varepsilon, F) = \{(0, a)\}$$

$$4 : \delta(0, a, a) = \{(0, \varepsilon)\}$$

Fita:$[0, a * (a + a), E] \vdash [0, a * (0 + a), T] \vdash [0, a * (a + a), T * F] \vdash [0, a*(a+a), F*F] \vdash [0, a*(a+a), a*F] \vdash [0, *(a+a), *F] \vdash [0, (a+a), F] \vdash [0, (a + a), (E)] \vdash$

Criação de novas funções:

$$1 : \delta(0, \varepsilon, E) = \{(0, T)\}$$

$$2 : \delta(0, \varepsilon, T) = \{(0, T * F), (0, F)\}$$

$$3 : \delta(0, \varepsilon, F) = \{(0, a)\}$$

$$4 : \delta(0, a, a) = \{(0, \varepsilon)\}$$

$$5 : \delta(0, *, *) = \{(0, \varepsilon)\}$$

$$6 : \delta(0, (, () = \{(0, \varepsilon)\}$$

Fita:$[0, a * (a + a), E] \vdash [0, a * (0 + a), T] \vdash [0, a * (a + a), T * F] \vdash [0, a*(a+a), F*F] \vdash [0, a*(a+a), a*F] \vdash [0, *(a+a), *F] \vdash [0, (a+a), F] \vdash [0, (a + a), (E)] \vdash [0, a + a), E)] \vdash [0, a + a), E + T)] \vdash [0, a + a, T + T)] \vdash [0, a + a), F + T)] \vdash [0, a + a), a + T)] \vdash [0, +a), +T)] \vdash$

Criação da regra para + no inicio da fita e + no topo da pilha:

$$1 : \delta(0, \varepsilon, E) = \{(0, T)\}$$

$$2 : \delta(0, \varepsilon, T) = \{(0, T * F), (0, F)\}$$

$$3 : \delta(0, \varepsilon, F) = \{(0, a)\}$$

$$4 : \delta(0, a, a) = \{(0, \varepsilon)\}$$

$$5 : \delta(0, *, *) = \{(0, \varepsilon)\}$$

$$6 : \delta(0, (, () = \{(0, \varepsilon)\}$$

$$7 : \delta(0, +, +) = \{(0, \varepsilon)\}$$

Fita:$[0, a * (a + a), E] \vdash [0, a * (0 + a), T] \vdash [0, a * (a + a), T * F] \vdash [0, a*(a+a), F*F] \vdash [0, a*(a+a), a*F] \vdash [0, *(a+a), *F] \vdash [0, (a+a), F] \vdash [0, (a + a), (E)] \vdash [0, a + a), E)] \vdash [0, a + a), E + T)] \vdash [0, a + a, T + T)] \vdash [0, a+a), F+T)] \vdash [0, a+a), a+T)] \vdash [0, +a), +T)] \vdash [0, a), T)] \vdash [0, a), F)] \vdash [0, a), a)] \vdash [0,),)] \vdash$

Criação de nova regra:

$$1 : \delta(0, \varepsilon, E) = \{(0, T), (0, E + T)\}$$

**em algum momento foi adicionado*

$$2 : \delta(0, \varepsilon, T) = \{(0, T * F), (0, F)\}$$

$$3 : \delta(0, \varepsilon, F) = \{(0, a), (0, (E)\}$$

**em algum momento foi adicionado*

$$4 : \delta(0, a, a) = \{(0, \varepsilon)\}$$

$$5 : \delta(0, *, *) = \{(0, \varepsilon)\}$$

$$6 : \delta(0, (, () = \{(0, \varepsilon)\}$$

$$7 : \delta(0, +, +) = \{(0, \varepsilon)\}$$

$$8 : \delta(0,),)) = \{(0, \varepsilon)\}$$

Fita:$[0, a * (a + a), E] \vdash [0, a * (0 + a), T] \vdash [0, a * (a + a), T * F] \vdash [0, a*(a+a), F*F] \vdash [0, a*(a+a), a*F] \vdash [0, *(a+a), *F] \vdash [0, (a+a), F] \vdash [0, (a + a), (E)] \vdash [0, a + a), E)] \vdash [0, a + a), E + T)] \vdash [0, a + a, T + T)] \vdash [0, a+a), F+T)] \vdash [0, a+a), a+T)] \vdash [0, +a), +T)] \vdash [0, a), T)] \vdash [0, a), F)] \vdash [0, a), a)] \vdash [0,),)] \vdash [0, \varepsilon, \varepsilon \vdash^*$

3 Tarefa com a mesma gramática

Fita:
$[0, a * ((a + (a * a)), E] \vdash$

3.1 Tarefa: montar o conjunto de funções de transição

Gramática
$S ::=< A >< S > |\varepsilon$
$A ::=$ se *cond* então $< B >$ | enquanto *cond*$\{< C >\}$
$B ::= \{< C >\}|\{< C >\}$ else $\{< C >\}$
$C ::=< A >$ | exp

4 Análise Preditiva Tabular

O analisador preditivo precisa do conjunto *follow*, por isso usamos o símbolo de final de sentença. X = topo da pilha. N = conjunto dos não terminais. Quando aparecer um V maiúsculo é um terminal v (OR).

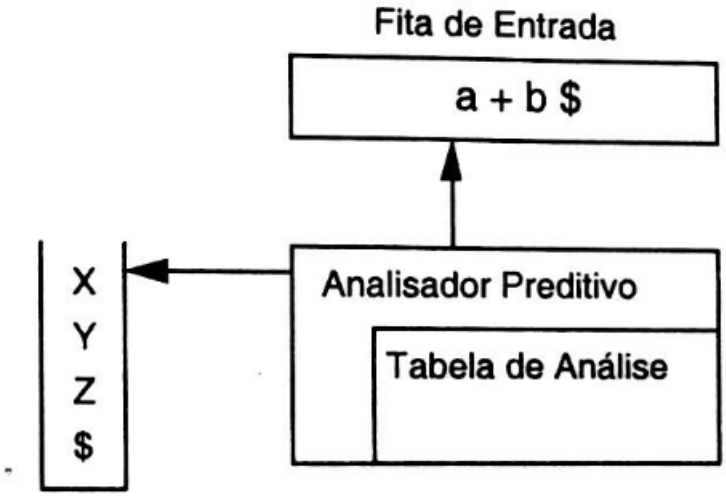


Figura 3.4 Analisador preditivo tabular

1. se $X = a = \$$, aceita
2. se $X = a \neq \$$
   - desempilha a
   - avança posição de leitura na fita
3. se $X \in N$
   - consulta $M[X, a]$
   - se $M[X, a] = \{X ::= UVW\}$
   - se $M[X, a] = \emptyset$, erro

A tabela de análise preditiva para essa gramática é mostrada a seguir:

	id	v	&	¬	$
E	E → T E'			E → T E'	
E'		E' → vT E'			E' → ε
T	T → F T'			T → F T'	
T'		T' → ε	T' → & F T'		T' → ε
F	F → id			F → ¬ F	

Com a sentença **idvid&id**, o reconhecedor preditivo realiza os movimentos mostrados na Figura 3.5.

substituir id por a

$E ::= EvT|T$
$T ::= T\&F|F$
$F ::= \neg F|a$
O analisador preditivo não reconhece recursão à esquerda, portanto vamos retirar a recursão.
$E ::= TE'$
$E' ::= VTE'|\varepsilon$
$T ::= FT'$
$T' ::= \&FT'|\varepsilon$
$F ::= \neg F|a$
Essa gramática é recursiva à direita e não à esquerda.

o inicio do topo da pilha é definido pelo símbolo inicial da gramática.

Pilha $E\$$	Entrada da fita	ação
$\$TE'\$$	$ava\&a\$$	$E ::= TE'$
$FT'E'\$$	$ava\&a\$$	$T ::= FT'$
$aT'E'\$$	$ava\&a\$$	$F :: a$
$T'E'\$$	$va\&a\$$	$T' :: \varepsilon$
$E'\$$	$va\&a\$$	$E' ::= VTE'$
$VTE'\$$	$va\&a\$$	$T' ::= FT'$
$TE'\$$	$a\&a\$$	
$FT'E'\$$	$a\&a\$$	$F ::= a$
$aT'E'\$$	$a\&a\$$	$T' ::= \&FT'$
$T'E'\$$	$\&a\$$	
$\&FT'E'\$$	$\&a\$$	$T' ::= \&FT'$
$FT'E'\$$	$a\$$	$F ::= a$
$aT'E'\$$	$a\$$	
$T'E'\$$	$\$$	$T' ::= \varepsilon$
$E'\$$	$\$$	$E' :: \varepsilon$
$\$$	$\$$	Aceite

Tabela first e follow::

	First	Follow
E	$\neg, a$	$\$$
E'	V, ε	$\$$
T	$\neg, a$	$V, \$$
T'	$\&, \varepsilon$	$V, \$$
F	$\neg, a$	$\&, V, \$$

Para acrescentar uma ação na tabela preditiva, a produção será acrescentada na linha definida pelo símbolo que dá nome à regra nas colunas do first da produção. Quando uma produção é ε, acrescentamos a produção na linha do símbolo que dá nome à regra nas colunas definidas pelo follow do mesmo símbolo.

4.1 Tarefa com a gramática do TF1

Gramática

$S ::= AS|\varepsilon$
$A ::=$ se *cond* então $B|$ enquanto *cond*$\{C\}$
$B ::= \{C\}|\{C\}$ senao $\{C\}$
$C ::= A|$ exp

Eliminação da recursão à esquerda: Não existe nenhuma recursão à esquerda nessa gramática.

Construção da tabela de análise preditiva:

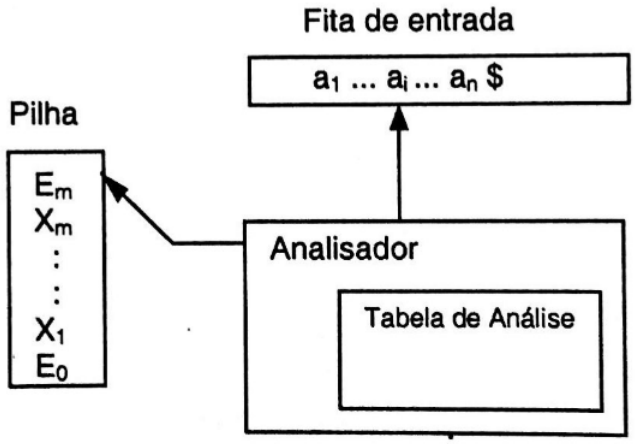
Gramática Fatorada:

$S ::= AS|\varepsilon$
$A ::=$ se $cond$ então $B|$ enquanto $cond\{C\}$
$B ::= \{C\}B'$
$B' ::=$ senao $\{C\}|\varepsilon$
$C ::= A|$ exp

	se	*cond*	*entao*	*enquanto*	{	}	*senao*	*exp*	$
S	$S \to AS$			$S \to AS$					$S \to \varepsilon$
A	$A \to$se cond entao B			$A \to$ enquanto cond $\{C\}$					
B					$B \to \{C\}B'$				
B'	$B' \to \varepsilon$			$B' \to \varepsilon$		$B' \to \varepsilon$	$B' \to$ senao $\{C\}$		$B' \to \varepsilon$
C	$C \to A$			$C \to A$				$C \to exp$	

5 Analisadores LR

- redutores (ascendentes)
- esquerda → direita (**derivação** mais à direita)
- estruturas sintáticas LLC
- identificação de erros (lookahead LALR)
- exige mais esforço de implementação
- SLR
- LALR (precisa adicionar a transição da próxima transição e da seguinte)
- LR
- utiliza a pilha



X_j - símbolo da gramática

E_j - estado

Figura 3.13 Estrutura dos analisadores LR

5.1 Ações da tabela de análise

- Empilha
- Reduz (sempre é seguida por um salto)

- Salto
- Aceite (reconhecimento e acontece em uma situação só da tabela)
- Erro (identificação e tratamento)

5.2 Passos de construção de um analisador LR

- GLC
- Itens válidos (canônicos)
- Transições pelos terminais (from itens válidos)
- Saltos pelos não-terminais, depois de uma redução(from itens válidos)
- First/Follow
- Construção da Tabela de Análise a partir de:
  1. Estados
  2. Transições
  3. Saltos
  4. Follow
  5. Identificador das produções
- Algoritmo de reconhecimento
- ID das produções

5.3 Construção do conjunto de itens válidos:

$S ::= a|[L]$
$L ::= L; S|S$

Iniciar pelo estado 0. *Todas as produções do estado inicial precisam aparecer no estado 0*

$Io = \{$

Itens válidos:
$S' ::= S\$$
$S ::= a|[L]$
$L ::= L; S|S$
$I_0 = \{S' ::= \bullet S\$, S ::= \bullet a, S ::= \bullet[L]\}$
$S' \rightarrow \bullet$

1-Transição pelo mesmo Sb no estado de origem?
2-Estado corrente (•) precede um NT? Se não, não precisa copiar as regras desse não terminal
$I_1 = \{S' ::= S \bullet \$\}(Completo!)$
$I_2 = \{S ::= a\bullet\}(Completo!)$
3-Estado Corrente (•) está no final de algum item válido(produção)? Se sim, esse estado possui um item chamado de "completo" que vai resultar numa ação de redução seguida de um salto.
$I_3 = \{S ::= [\bullet L], L ::= \bullet L; S, L ::= \bullet S, S ::= \bullet a, S ::= \bullet [L]\}$
$I_4 = \{S ::= [L\bullet], L ::= L\bullet; S\}$
$I_5 = \{L ::= S\bullet\}(Completo!)$
$I_6 = \{S ::= [L]\bullet\}(Completo!)$
$I_7 = \{L ::= L; \bullet S, S ::= \bullet a, S ::= \bullet [L]\}$
$I_8 = \{L ::= L; S\bullet\}$ (Completo!)

5.4 Construção das transições:

Partindo de um estado, olhar para o item e o estado corrente dele e identificar o estado em que o item tenha transição. Quando a transição ocorre com um símbolo não terminal, temos um salto, quando é por um terminal, temos um empilhamento.
$T(I_0, S) = I_1$
$T(I_0, a) = I_2$
$T(I_0, [) = I_3$
$T(I_3, L) = I_4$
$T(I_3, S) = I_5$
$T(I_3, a) = I_2$
$T(I_3, [) = I_3$
$T(I_4,]) = I_6$
$T(I_4, ;) = I_7$
$T(I_7, S) = I_8$
$T(I_7, a) = I_2$
$T(I_7, [) = I_3$

6 Construir conjunto de itens válidos, transições e saltos

Gramática:
$E ::= E + T | \bullet T$
$T ::= T * F | F$
$F ::= (E) | a$
$I_0 = \{E' ::= \bullet E\$, E ::= \bullet E + T, E ::= \bullet T, T ::= \bullet T * F, T ::= \bullet F, F ::= \bullet(E), F ::= \bullet a\}$
$I_1 = \{E' ::= E \bullet \$, E ::= E \bullet +T\}$
$I_2 = \{E ::= T\bullet, T ::= T \bullet *F\}$ (Completo)
$I_3 = \{T ::= F\bullet\}$ (Completo)
$I_4 = \{F ::= (\bullet E), E ::= \bullet E + T, E ::= \bullet T, T ::= \bullet T * F, T ::= \bullet F, F ::= \bullet(E), F ::= \bullet a\}$
$I_5 = \{F ::= a\bullet\}$ (Completo)
$I_6 = \{E ::= E + \bullet T, T ::= \bullet T * F, T ::= \bullet F, F ::= \bullet(E), F ::= \bullet a\}$
$I_7 = \{T ::= T * \bullet F, F ::= \bullet(E), F ::= \bullet a\}$
$I_8 = \{F ::= (E\bullet), E ::= E \bullet +T\}$
$I_9 = \{E ::= E + T\bullet, T ::= T \bullet *F,$ (Completo)
$I_{10} = \{T ::= T * F\bullet\}$ (Completo)
$I_{11} = \{F ::= (E)\bullet\}$ (Completo)

7 Construção da Tabela SLR

Para construir a Tabela SLR precisamos do conjunto de transições, conjunto de saltos, conjunto First e conjunto Follow.

Gramática:
$E' :: E\$$
$E ::= E + T | \bullet T$
$T ::= T * F | F$
$F ::= (E) | a$
Tabela first e follow:

	First	Follow
E	$(, a$	$\$, +,)$
T	$(, a$	$*, \$, +,)$
F	$(, a$	$*, \$, +,)$

Conjunto de Transições e Saltos:
$T(I_0, E) = I_1, T(I_0) = I_2, T(I_0, F) = I_3, T(I_0, () = I_4, T(I_0, a) = I_5$
$T(I_1, +) = I_6$

$T(I_2, *) = I_7$
$T(I_4, *) = I_8, T(4, T) = I_2, T(I_4, F) = I_3, T(I_4, () = I_4, T(I_4, a) = I_5$
$T(I_6, T) = I_9, T(I_6, F) = I_3, T(I_6, () = I_4, T(I_7, a) = I_5$
$T(I_7, F) = I_{10}, T(I_7, () = I_4, T(I_7, a) = I_5$
$T(I_8,)) = I_{11}, T(I_8, +) = I_6$
$T(I_9, *) = I_7$

Tabela sem operações de redução

	a	+	*	(	)	$	E	T	F
0	e5			e4			1	2	3
1		e6							
2			e7						
3									
4	e5			e4			8	2	3
5									
6	e5			e4				9	3
7	e5			e4					10
8		e6			e11				
9			e7						
10									
11									

A operação de redução é apenas para o item completo. Para o estado 2, para $E :: T$, nós vamos adicionar na tabela a Redução. Lembrando que o número da redução é referente à ordem das produções da gramática.

```
para I2
  para E::=T
  add R2
  linha 2
  colunas do Follow (E)


para I3
 para
```

	a	+	*	(	)	$	E	T	F
0	e5			e4			1	2	3
1		e6							
2		r2	e7		r2	r2			
3		r4			r4	r4			
4	e5			e4			8	2	3
5		r6	r6		r6	r6			
6	e5			e4				9	3
7	e5			e4					10
8		e6			e11				
9		r1	e7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

Incluir Aceite no final de sentença

	a	+	*	(	)	$	E	T	F
0	e5			e4			1	2	3
1		e6				AC			
2		r2	e7		r2	r2			
3		r4			r4	r4			
4	e5			e4			8	2	3
5		r6	r6		r6	r6			
6	e5			e4				9	3
7	e5			e4					10
8		e6			e11				
9		r1	e7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

8 Construir a tabela SLR

Gramática

$S ::=< A >< S > |\varepsilon$
$A ::=$ se *cond* então $< B >$ | enquanto *cond*$\{< C >\}$
$B ::= \{< C >\}|\{< C >\}$ else $\{< C >\}$
$C ::=< A >$ | exp